Table of Contents

Submission Request

Please do not submit pdf format throughout the course. I prefer Word format is possible.

Stateful Widgets

Stateful widgets can update their appearance in response to user interactions or data changes. While the widget itself is immutable, it creates a separate **State** object to hold and manage state variables. Because the UI refreshes whenever the state is updated, stateful widgets tend to be more resource-intensive than stateless widgets. To create a Stateful widget, two objects are required:

1. A StatefulWidget.
2. A State Object.

Example 1: Stateful Widget Helloworld!

This example shows the basic dart code that is needed to generate a basic mobile screen with a Stateful widget and State object.

To build this project, create a new Flutter application. Then, replace the contents of **main.dart** with the following:

```dart
import 'package:flutter/material.dart';

// 1. The program launch is here.
void main() {
  runApp(new MaterialApp(home: new MyApp()));
}

// 2. This class extends from widget family which is immutable.
//    However, it can instantiate a state object though.
class MyApp extends StatefulWidget {
  @override
  MyStateFullApp createState() => new MyStateFullApp();// Launch stateful component.
}

// 3. The state object allows mutable properties.
class MyStateFullApp extends State<MyApp> {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Welcome to Flutter'),
        ),
        body: const Center(
          child: Text('Hello World'),
        ),
      ),
    );
  }
}
```

Exercise 1 (2 marks)

Fill in the blanks with one of the three terms for each statement.

main() **StatefulWidget** **State object**

A _____ manages state properties.

_____ is where the application begins.

A _____ is not changeable once it has been created.

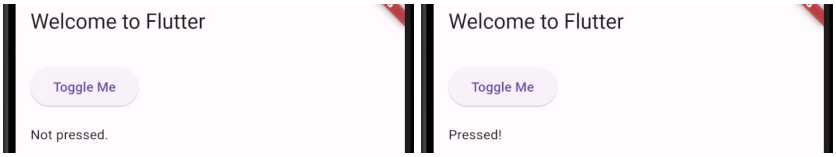
Exercise 2 (1 mark)

Why are stateful widgets are immutable? (Please examine the code comments).

State Properties (and Buttons)

Example 2: Buttons

This example demonstrates how the state property named *_text* is set in the State object.



To build this example, replace the State object in Example 1with this code:

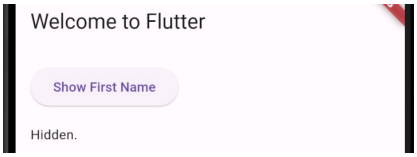
```
// 3. The state object allows mutable properties.
class MyStateFullApp extends State<MyApp> {
  static const PRESSED     = "Pressed!";
  static const NOT_PRESSED = "Not pressed.";

  // Underscore ensures that the member is private.
  String _text = NOT_PRESSED;

  void setTextContent() {
    setState(() {
      if(_text==PRESSED) {
        _text = NOT_PRESSED;
      }
      else {
        _text = PRESSED;
      }
    });
  }

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Welcome to Flutter'),
        ),
        body: Padding(
          padding: EdgeInsets.all(16.0), // Add padding around the Column
          child: Column(
            // Ensure column children are left-aligned
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              ElevatedButton(
                onPressed: () {
                  setTextContent();
                },
                child: Text("Toggle Me",
                  textAlign: TextAlign.left,),
              ),
              SizedBox(height: 16), // Add space between button and text
              Text(_text,  textAlign: TextAlign.left),
            ],
          ),
        ),
      ),
    );
  }
}
```

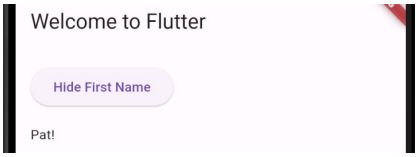
Exercise 3 (1 mark)

What is the role of the setState() function in the State object?

Exercise 4 (3 marks)

Adjust the app so that when the button labelled **Show First name** is clicked the first name appears and the button text changes. When the button labelled **Hide First Name** is clicked, the name becomes hidden and the button text changes accordingly like in the screenshots below. I used two state properties for this.



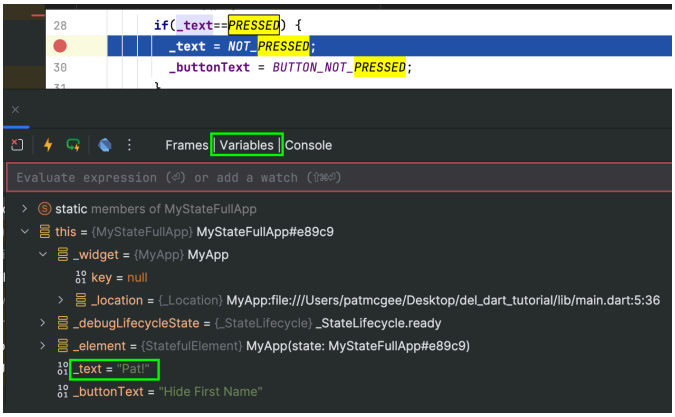


Show a screenshot of the output here but please use your name:

Show the code for MyStateFullApp here:

Exercise 5 (2 marks)

Take a screenshot while debugging and while halted in the setState() area within the condition when the button has been pressed. Be sure that your name appears in the display area within the screenshot.

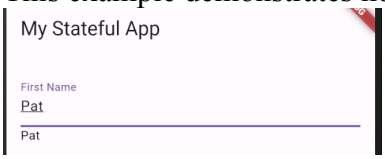


Show the screenshot here:

Stateful Object with TextField

Example 3: Adding a Text Field

This example demonstrates how to implement a *TextField* control within a State object.



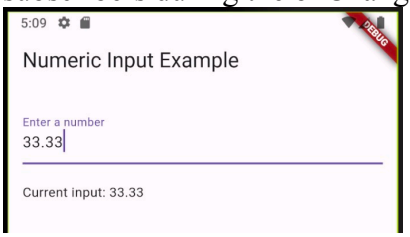
The *onChange()* handler re-assigns values to the *_firstName* variable with the *setState()* method. This assignment is possible because the control is contained within a stateful component. Here is the code for **lib\main.dart**:

```
import 'package:flutter/material.dart';

void main() {
  runApp(new MaterialApp(home: MyApp()));
}

class MyApp extends StatefulWidget {
  @override
  MyStateFullApp createState() => new MyStateFullApp();
}

class MyStateFullApp extends State<MyApp> {
  // Set text here.
  String _firstName = '';

  @override
  Widget build(BuildContext context) {
    return new Scaffold(
      appBar: new AppBar(
        title: new Text("My Stateful App"),
      ),
      body: Padding(
      padding: EdgeInsets.all(16.0), // Add padding around the Column
      child: Column(
        // Ensure column children are left-aligned
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
              new TextField(
                  onChanged: (String text){
                    setState(() {
                      _firstName = text;
                    });
                  },
                  decoration: new InputDecoration(
                  hintText: "First Name", labelText: "First Name")),
              new Text(_firstName),
            ],
        ),
      )
    );
  }
}
```

Custom Numeric-Only TextField

Example 4: Creating a Numeric-Only Text Widget

This example shows a reusable custom TextField stateless widget that sends its contents to subscribers during the onChange() event.



The following filter ensures that only numbers, decimals as well as + and – signs can be entered from the keyboard.

```
FilteringTextInputFormatter.allow(RegExp(r'^-?(([0-9]*)|([0-9]*)\.([0-9]*))$')),
```

The keyboardType property further limits the mobile users choices for entering text to help ensure inputs are valid:

```
keyboardType: TextInputType.numberWithOptions(decimal: true, signed: true),
```

Note that the contents on the input field is actually a String. The code for the reusable numeric input is contained inside numeric_input.dart:

lib\numeric_input.dart

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

class NumericInputField extends StatelessWidget {
  final Function(String) onChanged;
  // Constructor. Receives reference to callback function which has string param.
  NumericInputField({required this.onChanged});

  @override
  Widget build(BuildContext context) {
    return TextField(
      keyboardType: TextInputType.numberWithOptions(
        decimal: true,
        signed: true,
      ),
      inputFormatters: <TextInputFormatter>[
        FilteringTextInputFormatter.allow(
          RegExp(r'^-?(([0-9]*)|(([0-9]*)\.([0-9]*)))$'),
        ),
      ],
      decoration: InputDecoration(labelText: 'Enter a number'),
      onChanged: (String input) {
        onChanged(input);
      },
      // Sends text input to calling function.
    );
  }
}
```

After adding the numeric text input widget, replace the code inside main.dart with the following.

lib\main.dart

```
void main() {
  runApp(MaterialApp(home: MyHomePage()));
}

class MyHomePage extends StatefulWidget {
  // Create state allows widget to change UI with data changes & interactions.
  @override
  _MyHomePageState createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  String _inputValue = "";

  void _handleInputChange(String input) {
    setState(() {
      _inputValue = input;
    });
  }

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: Text('Numeric Input Example'),
        ),
        body: Padding(
          padding: const EdgeInsets.all(16.0),
          child: Column(
            // Ensure column children are left-aligned
            crossAxisAlignment: CrossAxisAlignment.start,

            children: [
              NumericInputField(onChanged: _handleInputChange),
              SizedBox(height: 20),
              Text('Current input: $_inputValue'),
            ],
          ),
        ),
      ),
    );
  }
}
```

Exercise 6 (2 marks)

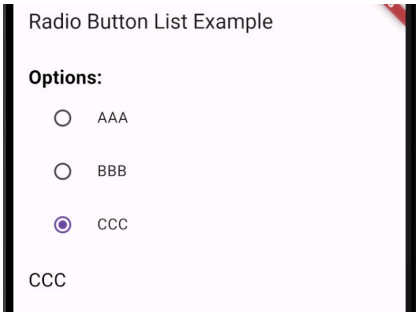
Create a re-usable StatelessWidget to store a TextField. The StatelessWidget must notify subscribers during each onChanged event. The StatelessWidget must also change the hint and label when initialized each time. For this case, the first object should display the First Name label and hint. The second object instance should display the Last Name label and hint.

Show your screenshot of your screen output here which looks like the screenshot above but with your name in it.

RadioListTile

Example 5: RadioListTile

This example demonstrates how to implement a custom RadioListTile in a stateless widget. The RadioListTile is set up so subscribing State objects can receive notification for each selected option's *onChanged* event.



Here the RadioListTile is encapsulated so in a separate class.

lib\my_radio_listtile.dart

```dart
import 'package:flutter/material.dart';

class MyRadioListTile<T> extends StatelessWidget {
  final List<String> options;
  final String selectedValue;
  final Function(String) onChanged;

  const MyRadioListTile({
    required this.options,   // Receives list of radio options.
    required this.onChanged, // Receives callback.
    required this.selectedValue,      // Receives the selected option string.
  });

  @override
  Widget build(BuildContext context) {
    print("\n***   MyRadioListTile has just been called. 'selectedValue'="
        + selectedValue.toString());

    return Column(
      children: options.map((option) {
        return RadioListTile<String>(
          value: option,
          title: Text(option.toString()),
          groupValue: selectedValue, // Show dot beside selected option.
          onChanged: (value) {
            if (value != null) {
              print("\n*** Inside onChanged for RadioListTile ***");
              print("'selectedValue'=" + this.selectedValue.toString());
              print("The value selected is: " + value.toString());
              onChanged(value); // Notify subscriber.
            }
          },
        );
      }).toList(),
    );
  }
}
```

The main screen is created with a stateful widget. The main screen draws the RadioListTile control and stores the selected value in a state variable called **selectedOption**. You will need to add a reference to **lib\MyRadioListTile.dart** – let the IDE do this for you by hovering over the error notice and selecting the file import.

lib/main.dart

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(MaterialApp(home: MyHomePage()));
}

class MyHomePage extends StatefulWidget {
  // Create state allows widget to change UI with data changes & interactions.
  @override
  _MyHomePageState createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  String selectedOption = "";

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Radio Button List Example')),
      body:Padding(padding: EdgeInsets.all(16.0),

    child: new Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: <Widget>[
            Text("Options: ",  style: new TextStyle(
              fontSize: 20.0,
              color: Colors.black,
              fontWeight: FontWeight.bold,
            ),
            ),
            MyRadioListTile<String>(
              options: ['AAA', 'BBB', 'CCC'],
              selectedValue: selectedOption,
              onChanged: (newValue) {
                print("\n*** Inside _MyHomePageState");
                setState(() {
                  print("    Updated value is: " + newValue);
                  selectedOption = newValue;
                });
              },
            ),
            SizedBox(height: 16),
            Text(selectedOption,  style: new TextStyle(
                fontSize: 20.0,
                color: Colors.black,
            ),
            )
          ]
```

Exercise 7 (2 marks)

Explain why selectedValue in MyRadioListTile is not updated right away as shown in the output?

```
***   MyRadioListTile has just been called. 'selectedValue'=BBB

*** Inside onChanged for RadioListTile ***
'selectedValue'=BBB
The value selected is: CCC

*** Inside _MyHomePageState
    Updated value is: CCC

***   MyRadioListTile has just been called. 'selectedValue'=CCC
```

Exercise 8 (1 mark)

Is MyRadioListTile a stateful or stateless object?

Basic Formatting in Flutter

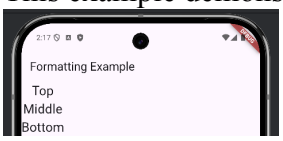
Formatting the Flutter screens nicely requires practice and experimentation – at least it took me a while to get used to formatting. To help get started, this section will discuss some formatting basics by covering code structures that are especially useful for presentation.

Columns and Rows

The Column widget places children vertically from top to bottom. The Row widget places children widgets horizontally (left to right).

Example 6: The Column Widget

This example demonstrates how the Column widget can arrange *Text* widgets horizontally.



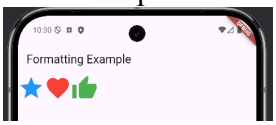
Here is the code with children Text widgets.

main.dart

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: const Text("Formatting Example")),
          body: Column(
            children: const [
              Text("Top",    style: TextStyle(fontSize: 24)),
              Text("Middle", style: TextStyle(fontSize: 24)),
              Text("Bottom", style: TextStyle(fontSize: 24)),
            ],
          ),
      ),
    );
  }
}
```

Example 7: The Row Widget

This example demonstrates how the Row widget can arrange icon widgets horizontally.



To build this example, replace the body of Example 6 with this body.

main.dart

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: const Text("Formatting Example")),
        body: Row(
          children: const [
            Icon(Icons.star, size: 50, color: Colors.blue),
            Icon(Icons.favorite, size: 50, color: Colors.red),
            Icon(Icons.thumb_up, size: 50, color: Colors.green),
          ],
        ),
    ),
    );
  }
}
```

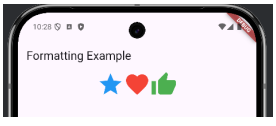
Main Axis Alignment

To help align the children widgets within a row, the **MainAxisAlignment** property can be set. Common options for MainAxisAlignment include:

```
MainAxisAlignment.start
MainAxisAlignment.center
MainAxisAlignment.end
MainAxisAlignment.spaceEvenly
MainAxisAlignment.spaceBetween
```

Example 8: MainAxisAlignmnet

This example shows how **MainAxisAlignment** can be used to center the children widgets within a **Row** widget.



To build this example, replace the body of Example 7 with this version.

```
body: Row(
  mainAxisAlignment:  MainAxisAlignment.center,
  children: const [
    Icon(Icons.star, size: 50, color: Colors.blue),
    Icon(Icons.favorite, size: 50, color: Colors.red),
    Icon(Icons.thumb_up, size: 50, color: Colors.green),
  ],
)
```

Exercise 9 (1 mark)

What single line of code in Example 8 is different than Example 7?

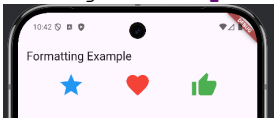
Exercise 10 (1 mark)

Make adjustments to Example 8 to position the children widgets at the right of the screen. Also, change the title so it is “Your first name’s formatting example”. Please use your actual first name to get the mark. Show a screenshot of the output after this change has been made:

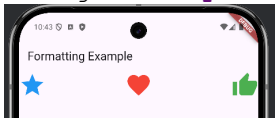
Example 9: Other MainAxisAlignment Options

Other options for mainAxisAlignment can lead to other positions.

`MainAxisAlignment.spaceEvenly`

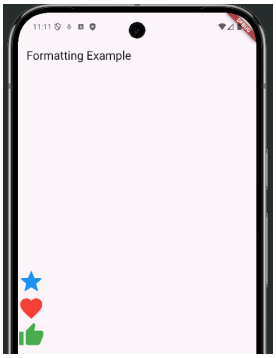


`MainAxisAlignment.spaceBetween`



Example 10: MainAxisAlignment Options for Columns

When MainAxisAlignment is used with Column instead of Row, the vertical alignment is adjusted. This example shows how. To build this example, replace the body of Example 7 with this version.



To build this example, replace the code in

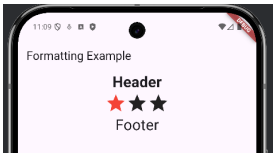
```
body: Column(
  mainAxisAlignment:  MainAxisAlignment.center,
  children: const [
    Icon(Icons.star, size: 50, color: Colors.blue),
    Icon(Icons.favorite, size: 50, color: Colors.red),
    Icon(Icons.thumb_up, size: 50, color: Colors.green),
  ],
)
```

Exercise 11 (1 mark)

What effect does `MainAxisAlignment.end` have when it is used inside a Column block versus a Row block?

Example 11: Combining Rows and Columns

This code combines rows and columns to create a more complex layout.

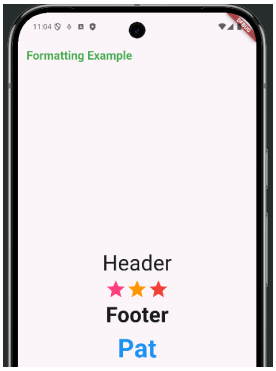


To build this example, replace the body in Example 7 with this version.

```
body: Column(
  children: [
    const Text("Header", style: TextStyle(fontSize: 28,
          fontWeight: FontWeight.bold)),
    Row(
      mainAxisAlignment: MainAxisAlignment.center,
      children: const [
        Icon(Icons.star, size: 40, color: Colors.red,),
        Icon(Icons.star, size: 40),
        Icon(Icons.star, size: 40),
      ],
    ),
    const Text("Footer", style: TextStyle(fontSize: 28)),
  ],
),
```

Exercise 12 (4 marks)

Modify Example 11 so the output is re-aligned, bolded and colored as shown. Also include your actual first name at the bottom to get this mark.



Show a screenshot of your output here:

Screen Overflow

With the Column widget alone, too many widgets may cause an overflow situation which leads to an unsightly output on the screen (see Figure 1).

Figure 1: Overflow error caused by too many widgets on the screen,

Also, when this overflow error occurs users will be unable to scroll to view widgets that are currently off the screen.

Example 12: Screen Overflow

This **main.dart** code produces the error seen in Figure 1.

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: const Text("List Without Scroll")),
        body: Column(
          children: [
            Text("Item 1", style: TextStyle(fontSize: 80)),
            Text("Item 2", style: TextStyle(fontSize: 80)),
            Text("Item 3", style: TextStyle(fontSize: 80)),
            Text("Item 4", style: TextStyle(fontSize: 80)),
            Text("Item 5", style: TextStyle(fontSize: 80)),
            Text("Item 6", style: TextStyle(fontSize: 80)),
            Text("Item 7", style: TextStyle(fontSize: 80)),
            Text("Item 8", style: TextStyle(fontSize: 80)),
            Text("Item 9", style: TextStyle(fontSize: 80)),
            Text("Item 10", style: TextStyle(fontSize: 80)),
            Text("Item 11", style: TextStyle(fontSize: 80)),
            Text("Item 12", style: TextStyle(fontSize: 80)),
          ],
        ),
      ),
    );
  }
}
```

Avoiding Screen Overflow with a Scrollable ListView

The overflow problem which occurs when too many widgets are displayed can be fixed by placing the widgets in a list that is contained within a ListView widget (see Figure 2).

Figure 2: Scrollable List with ListView

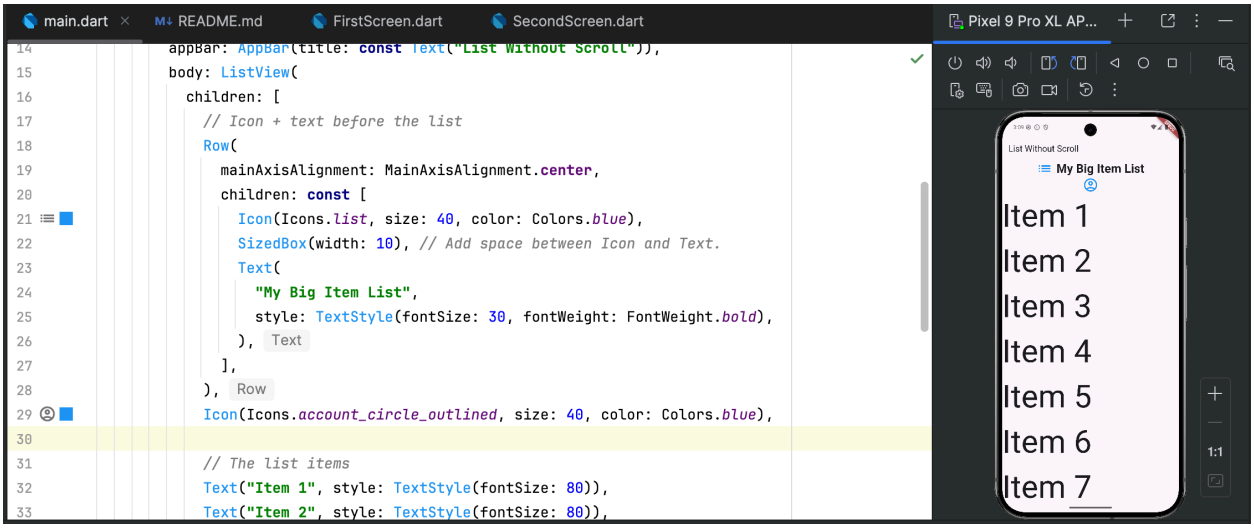
Example 13: Avoiding Screen Overflow with a Scrollable ListView

To build this example, replace the body in Example 12 with this version.

```
body: ListView(
  children: [
    // The list items
    Text("Item 1", style: TextStyle(fontSize: 80)),
    Text("Item 2", style: TextStyle(fontSize: 80)),
    Text("Item 3", style: TextStyle(fontSize: 80)),
    Text("Item 4", style: TextStyle(fontSize: 80)),
    Text("Item 5", style: TextStyle(fontSize: 80)),
    Text("Item 6", style: TextStyle(fontSize: 80)),
    Text("Item 7", style: TextStyle(fontSize: 80)),
    Text("Item 8", style: TextStyle(fontSize: 80)),
    Text("Item 9", style: TextStyle(fontSize: 80)),
    Text("Item 10", style: TextStyle(fontSize: 80)),
    Text("Item 11", style: TextStyle(fontSize: 80)),
    Text("Item 12", style: TextStyle(fontSize: 80)),
  ],
),
```

Example 14: Adding More Widgets

It is very likely that you will want to add more than just a list to your screen. There are many ways to do this. One way involves adding a Row widget. This example can be built by adding a Row widget to the children list in Example 13.

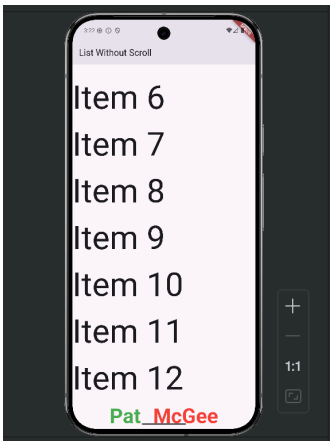


Here is the code:

```
// Icon + text before the list
Row(
  mainAxisAlignment: MainAxisAlignment.center,
  children: const [
    Icon(Icons.list, size: 40, color: Colors.blue),
    SizedBox(width: 10), // Add space between Icon and Text.
    Text(
      "My Big Item List",
      style: TextStyle(fontSize: 30, fontWeight: FontWeight.bold),
    ),
  ],
),
Icon(Icons.account_circle_outlined, size: 40, color: Colors.blue),
```

Exercise 13 (2 marks)

Display your first and last name after the 12th item in Example 14. Make sure your first and last name are different colours and that they appear in the same row. The name should look like the following:



Scroll to the bottom of the screen and take a screenshot of the content with your actual name after item 12. Show the entire phone screen in the screenshot.